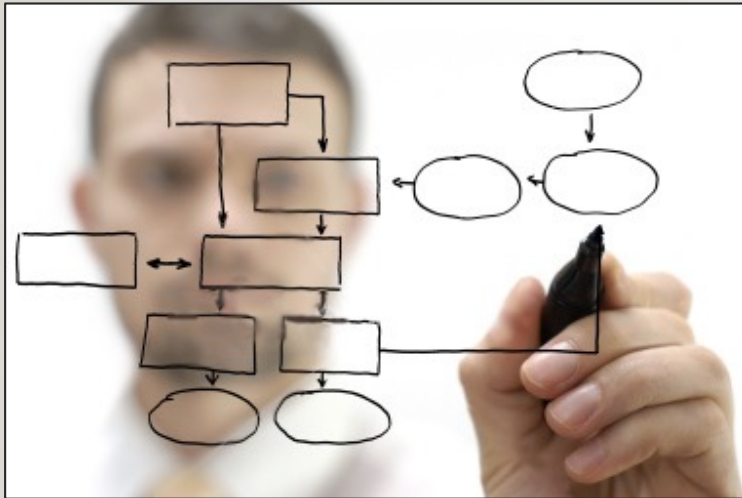


Documentación de Arquitecturas de Software



J. Andrés Díaz Pace

Diseño de Sistemas de Software

LA DEFINICIÓN DE ARQUITECTURA

*“The software architecture of a program or computing system is the **structure or structures of the system**, which comprise software components, the externally visible properties of those components, and the relationships among them”*

Documentación

*Software Architecture in Practice
(Bass, Clements and Kazman, 1998)*

- Todos los sistemas tienen una arquitectura
 - Querida o no, probablemente no muy documentada
 - Puede ser ambigua, informal, inconsistente
 - Apela a la descripción verbal (y ambigua!)

¿QUÉ Y CUÁNTO DOCUMENTAR?



CONTENIDOS DE LA DOCUMENTACIÓN

- Features y requerimientos funcionales
- Requerimientos de atributos de calidad (drivers)
- Restricciones
- **Architecturally-significant decisions**
- **Estructuras arquitectónicas** capturadas por medio de **vistas arquitectónicas**
- Comportamiento arquitectónico
- Interfaces (de componentes)
- Trazabilidad entre elementos



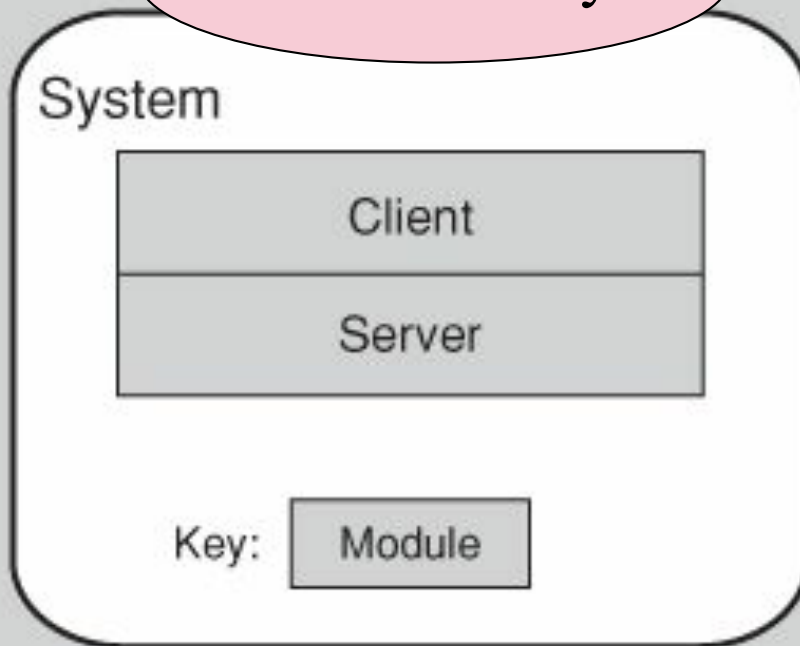
MÚLTIPLES ESTRUCTURAS

- Una arquitectura es una entidad multi-dimensional que usualmente es demasiado compleja como para ser vista/comprendida toda de una vez
- Los sistemas están compuestos por varias estructuras
 - Aspectos estáticos
 - Aspectos dinámicos
 - Correspondencia entre aspectos estáticos y dinámicos
 - Comportamiento



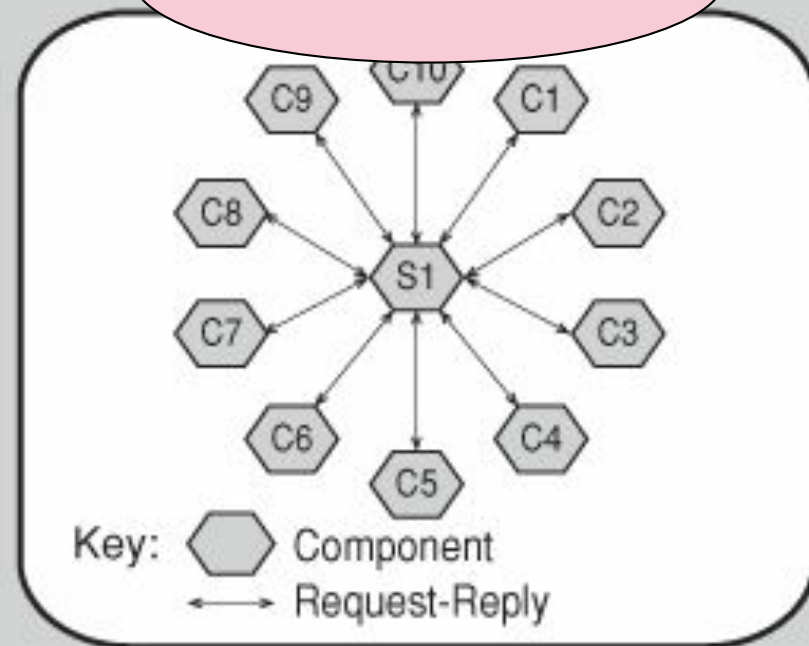
RELACIÓN ENTRE ESTRUCTURAS

Modifiability



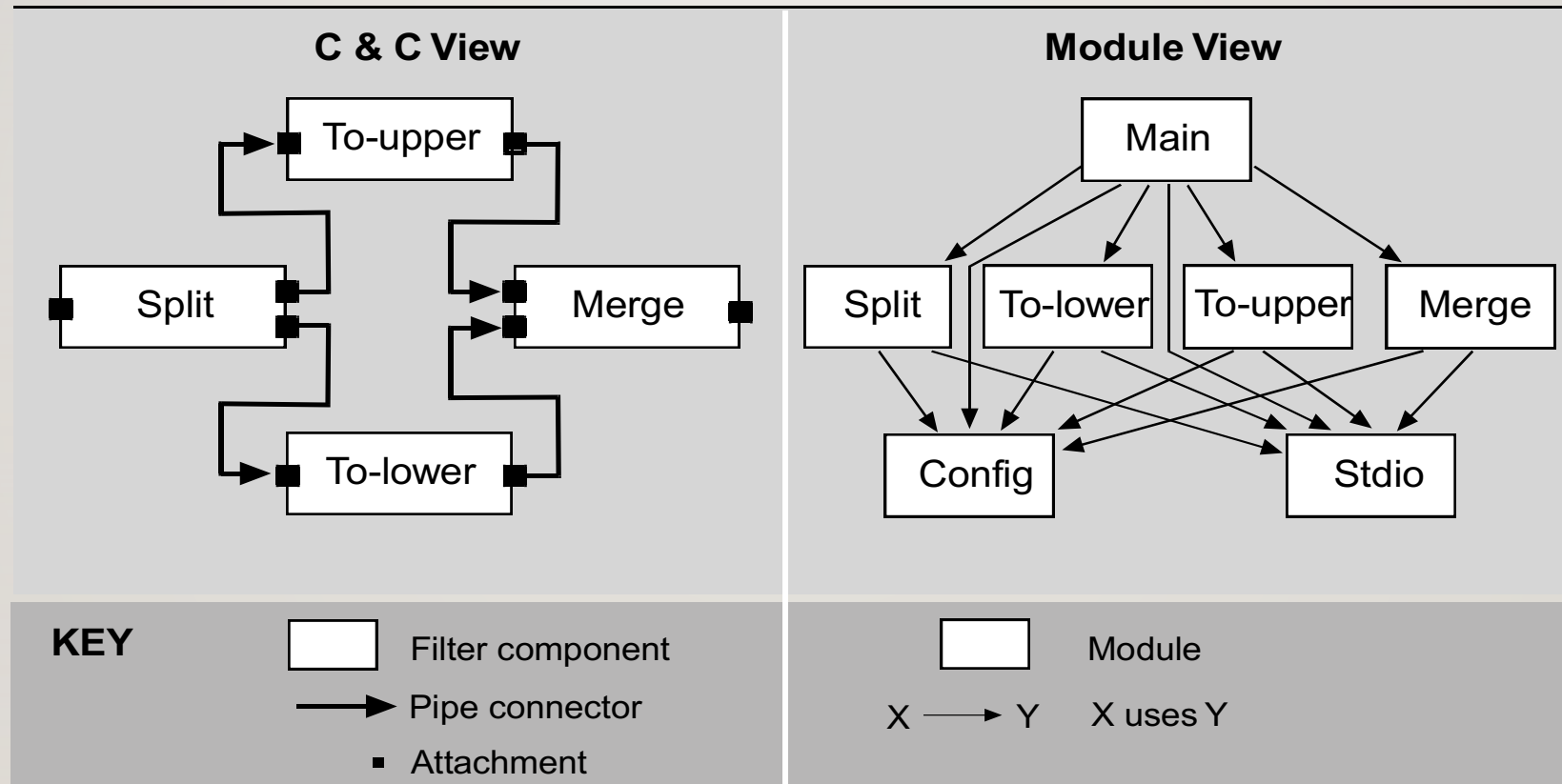
Decomposition View

Performance



Client-Server View

NO SIEMPRE ES UN MAPEO 1-1



3 VISTAS BÁSICAS

- Vista de **Módulos**

→ *construcción, análisis de impacto de cambios, testeo*

- Cómo se estructura en términos de un conjunto de unidades de código

- Vista de **Componentes y Conectores (C&C)**

→ *performance, disponibilidad*

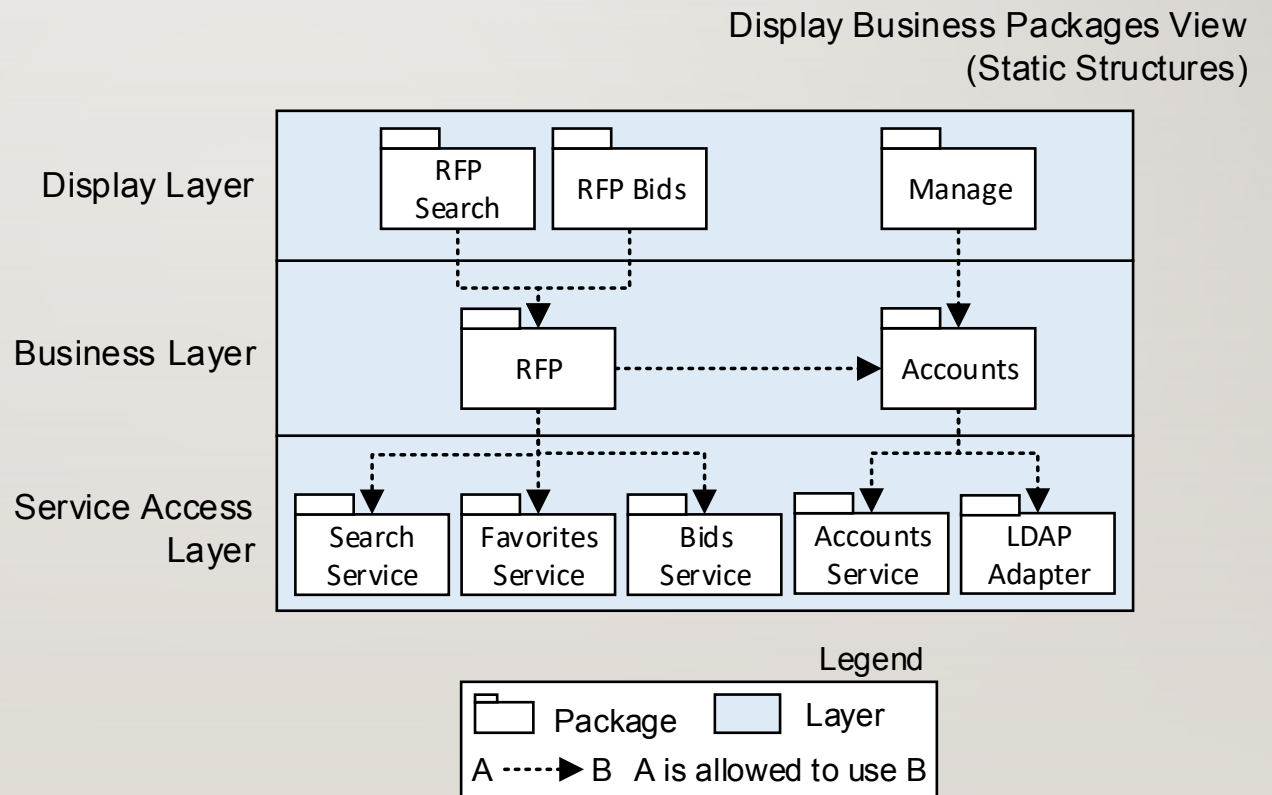
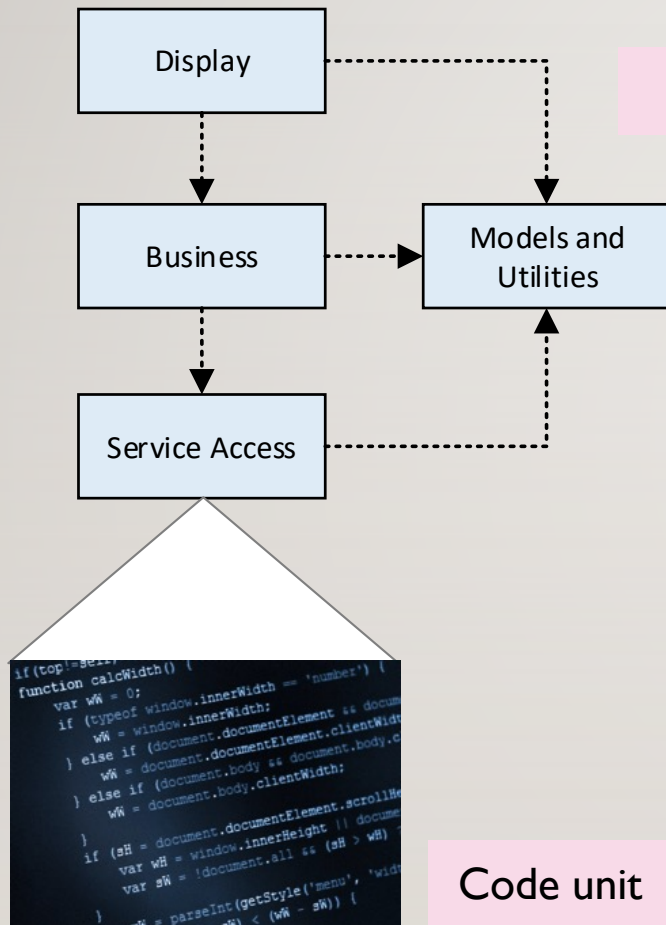
- Cómo se estructura en términos de un conjunto de elementos que poseen comportamiento en tiempo de ejecución e interacciones con otros elementos

- Vista de **Asignación (allocation)**

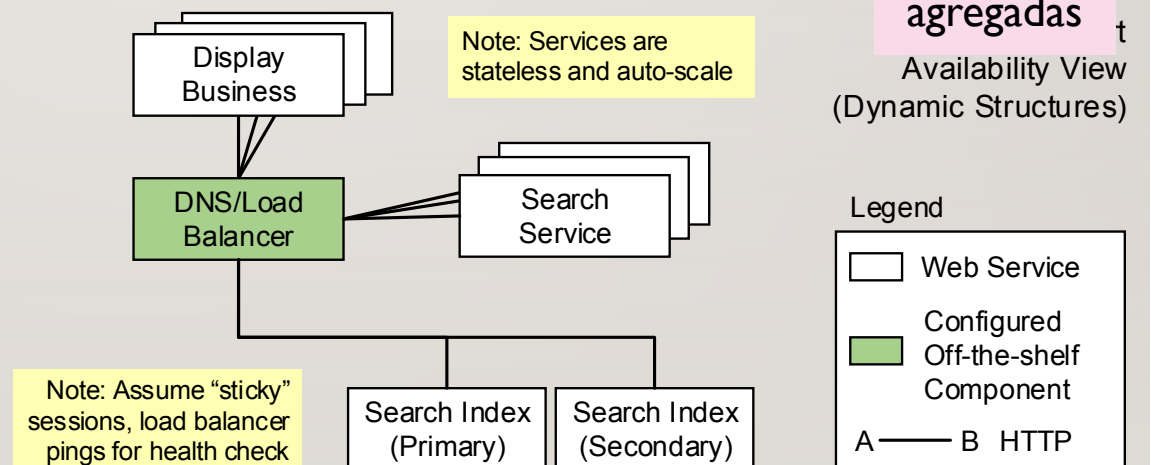
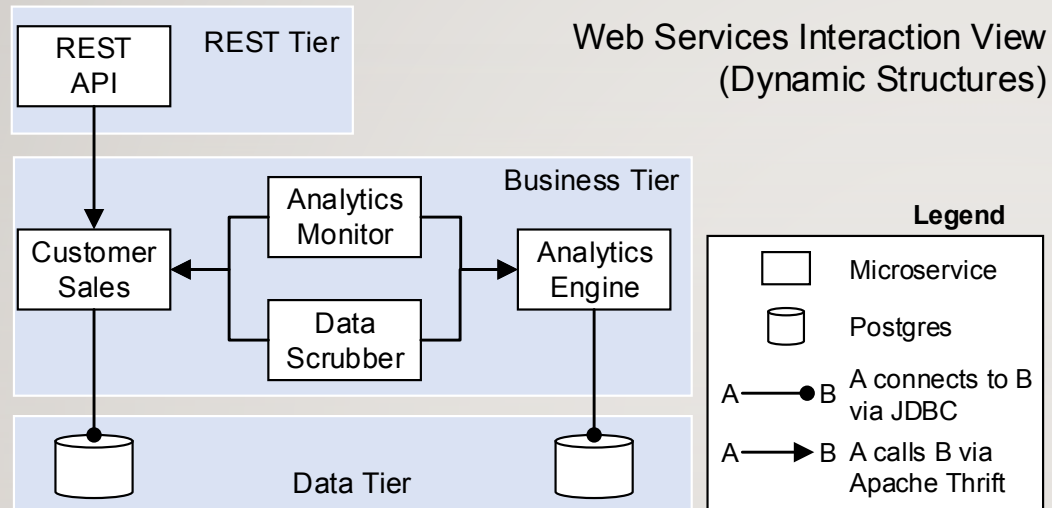
→ *disponibilidad, seguridad, performance*

- Como se relaciona con las estructuras no-software en su ambiente

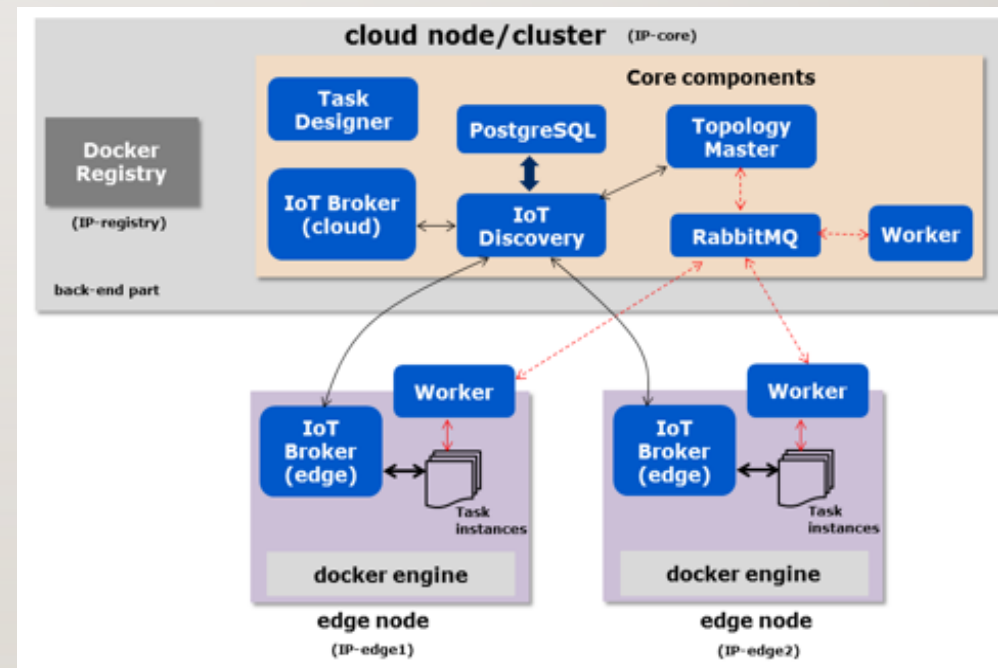
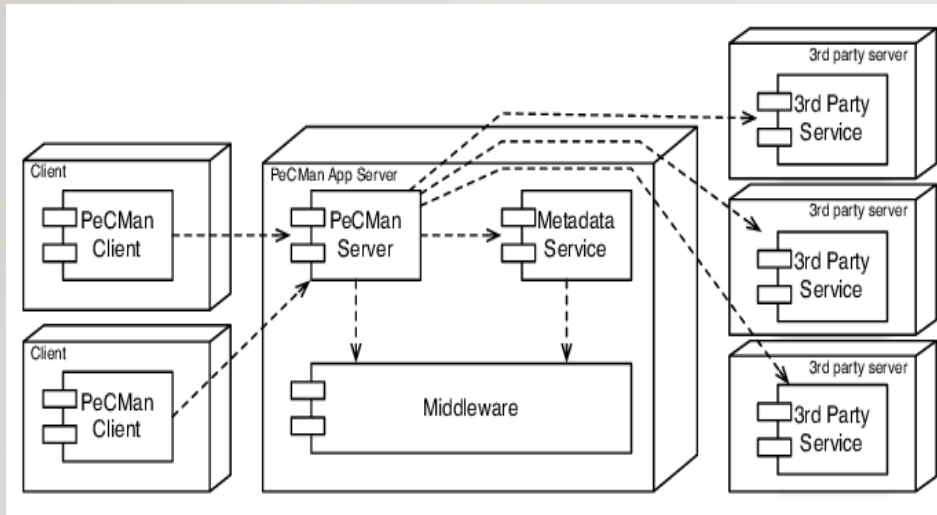
EJEMPLO DE VISTAS DE MÓDULOS (ESTÁTICA)



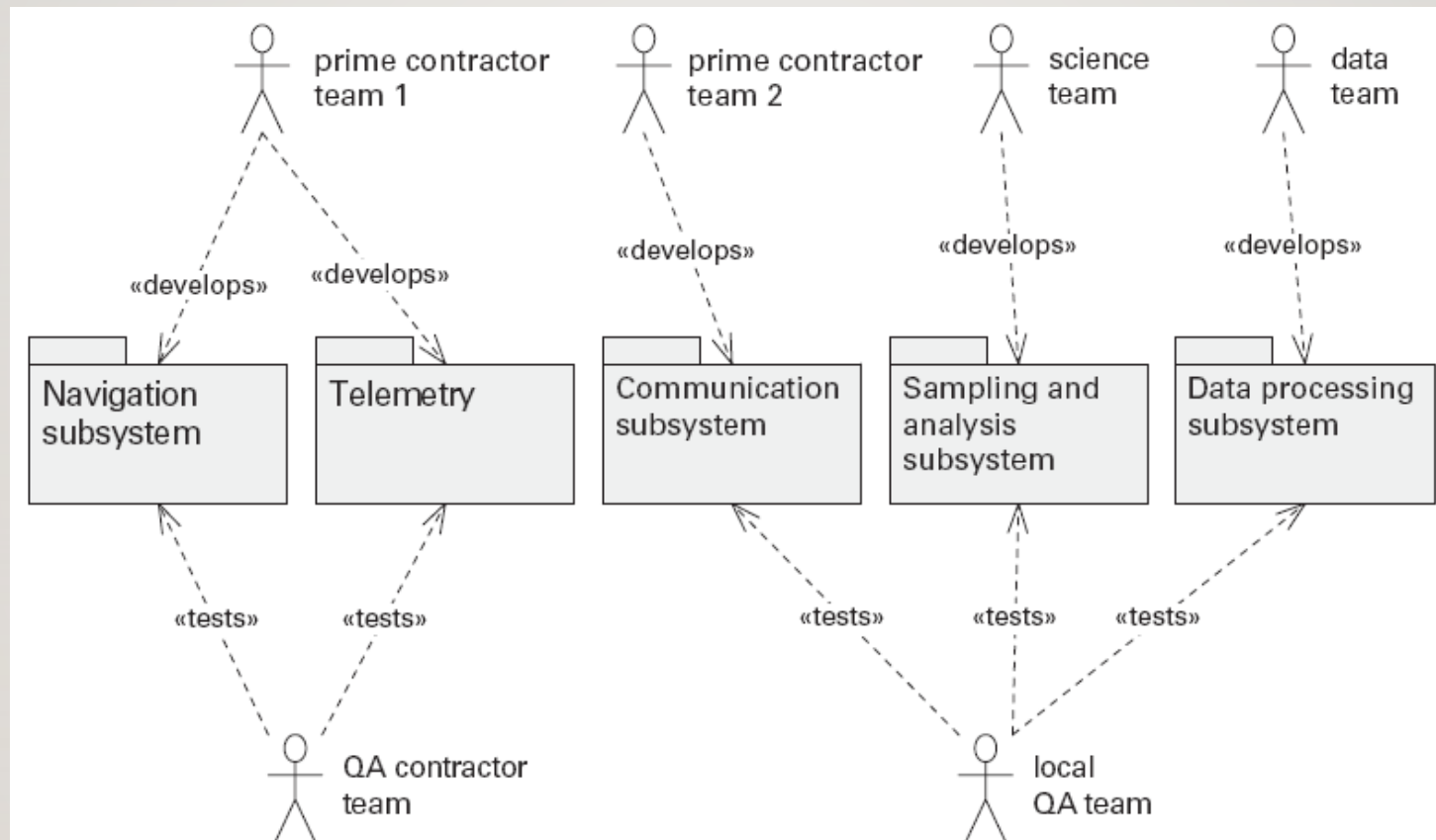
EJEMPLO DE VISTAS DE COMPONENTES (DINÁMICA)



EJEMPLOS DE VISTAS DE DESPLIEGUE



VISTA DE ASIGNACIÓN DE TRABAJO



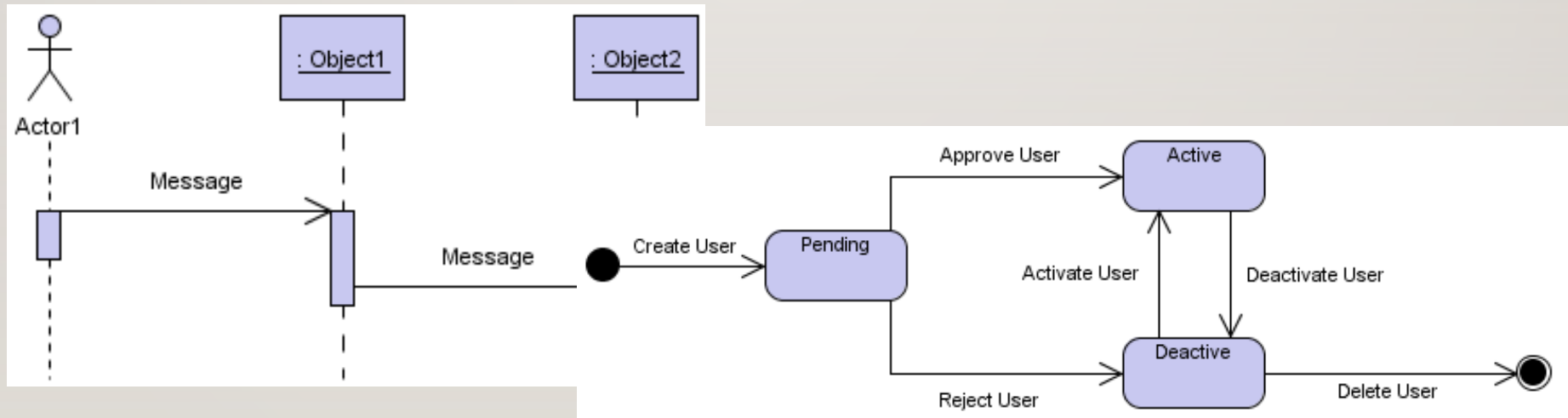
DOCUMENTACIÓN DE COMPORTAMIENTO

- Trace-Oriented

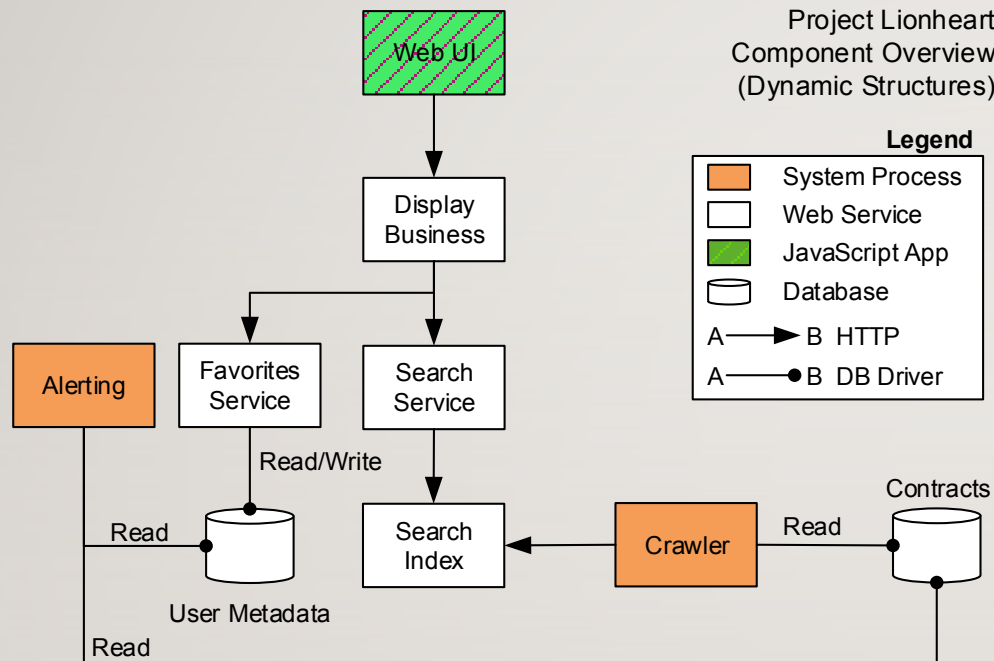
- use cases
- collaboration diagrams
- sequence diagrams
- message sequence charts
- activity diagrams
- ...

- Static

- Statecharts
- SDL diagrams
- Z specifications
- some ADLs
- ...



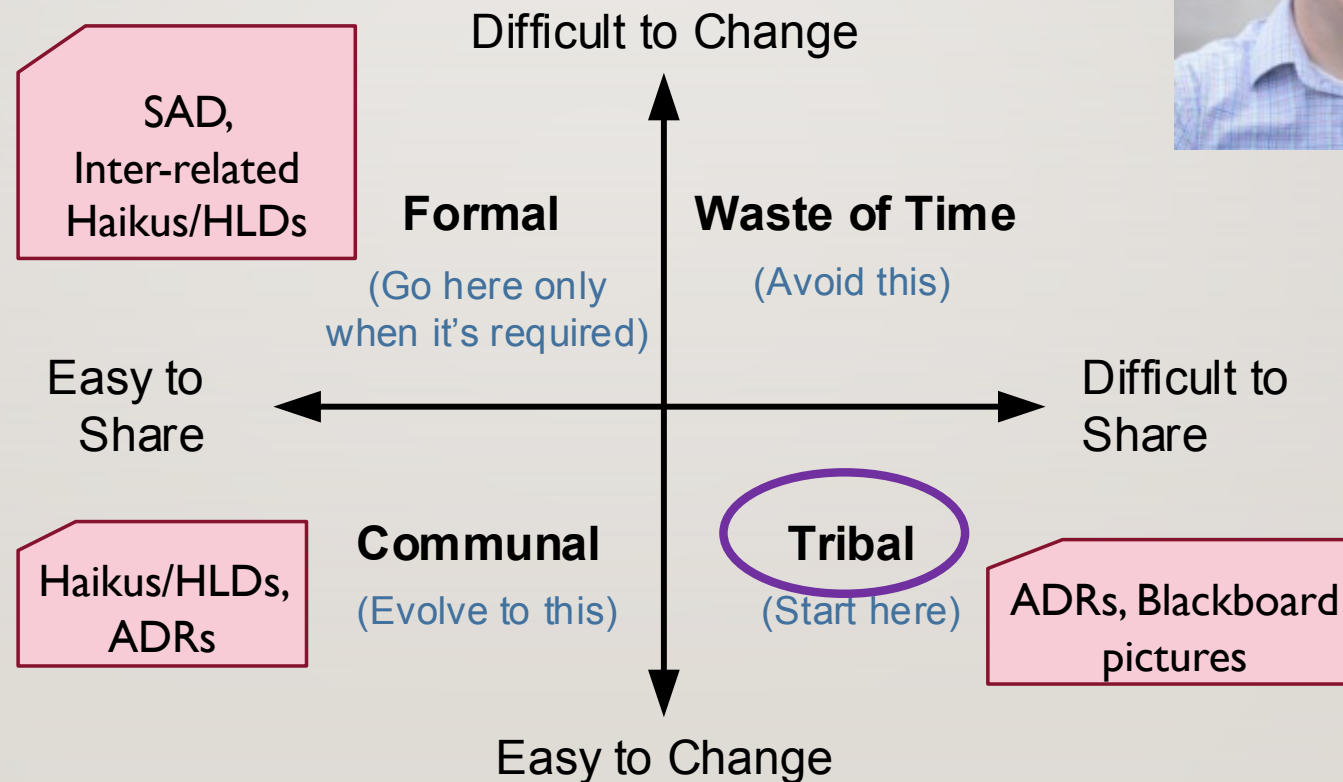
ASIGNACIÓN DE RESPONSABILIDADES



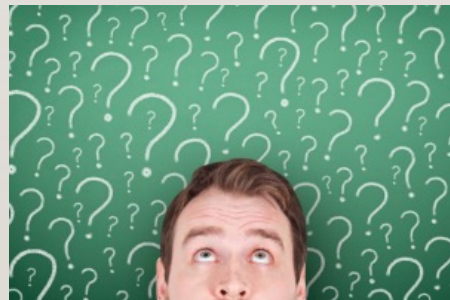
Element	Responsibility
Web UI	Renders a user interface for the user in their web browser, handles user interactions.
Display Business	Authentication and authorization, proxy for other backing services, verifies business logic for application use.
Search Service	Core processing for query parsing, search, pagination, filtering.
Favorites Service	Normalizes tags, writes favorites to persistent storage.
Alerting Service	Scheduled to look for recent changes, sends email based on subscriptions stored in user metadata database.
Crawler	Reads data from the Contracts DB, transforms it for search, uploads to index.
User Metadata DB	Persistent storage for user subscriptions and other user-added content.
Search Index	Optimized representation of contract data designed for search. All contract data to be displayed in the UI is searchable, sortable, and stored.
Contracts DB	Persistent storage. System of record for city RFP data.
HTTP Relation	Communication among services over standard HTTP protocols. APIs are assumed to be RESTful.
DB Driver Relation	Native driver/client for the to-be-selected database.

ESTRATEGIAS DE DOCUMENTACIÓN

- Depende de la audiencia (stakeholders) a la que se está apuntando con la documentación



SAD, HAIKUS, Y ADRs



George Fairbanks

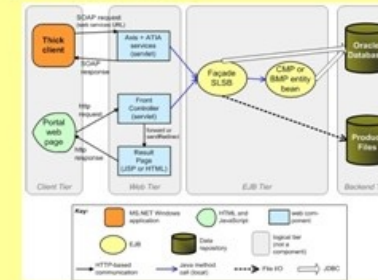
VIEWS & BEYOND TEMPLATES

1. Documentation Roadmap
2. How to document a view
3. System Overview
4. Views
 - 1. Link View 1
 - ...
 - n. Link View n
5. Mapping between Views
6. Mapping to Requirements
7. Rationale
8. Directory



Template for a View

- ## 1. Primary Presentation

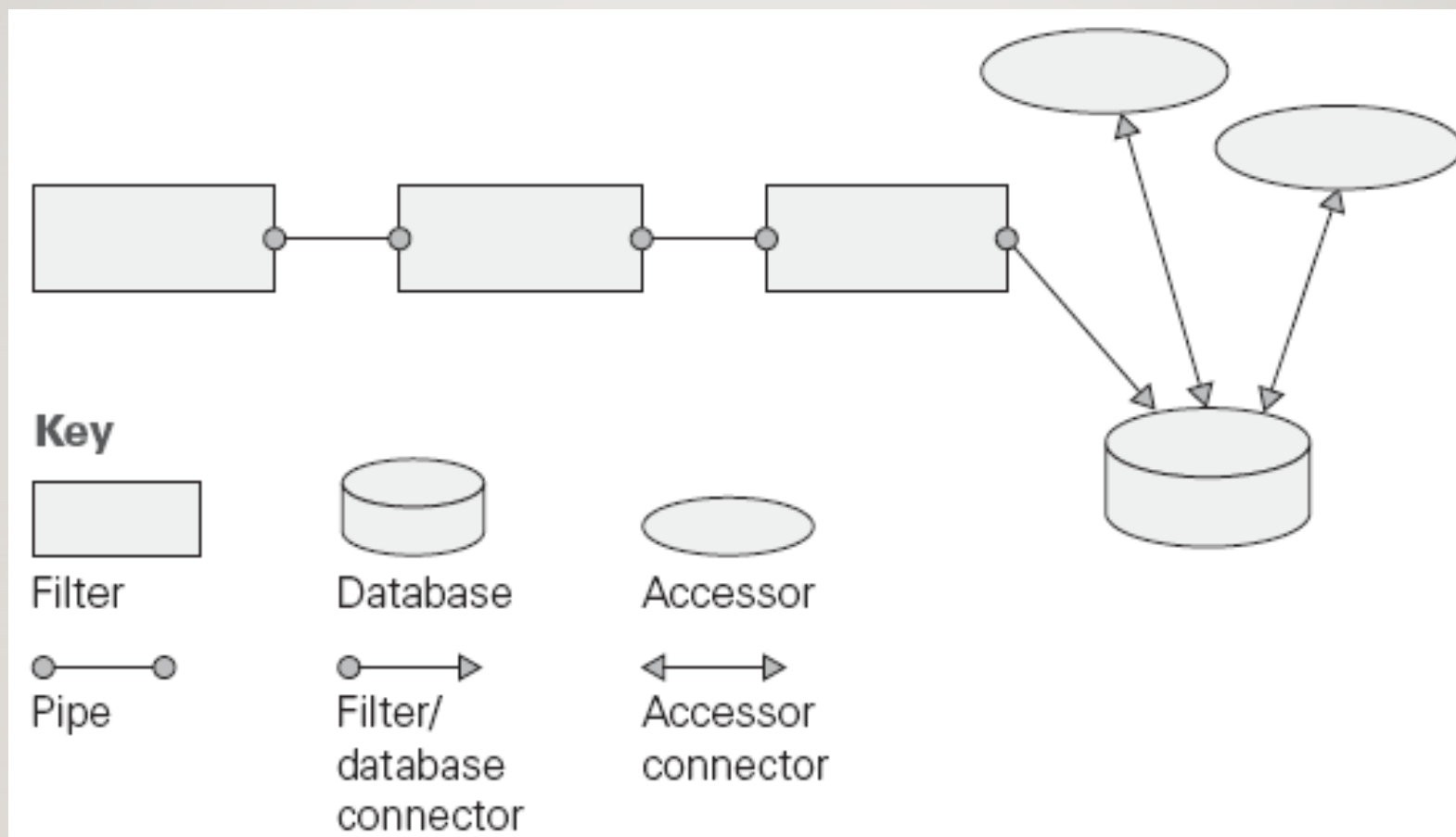


- ## 2. Element Catalog

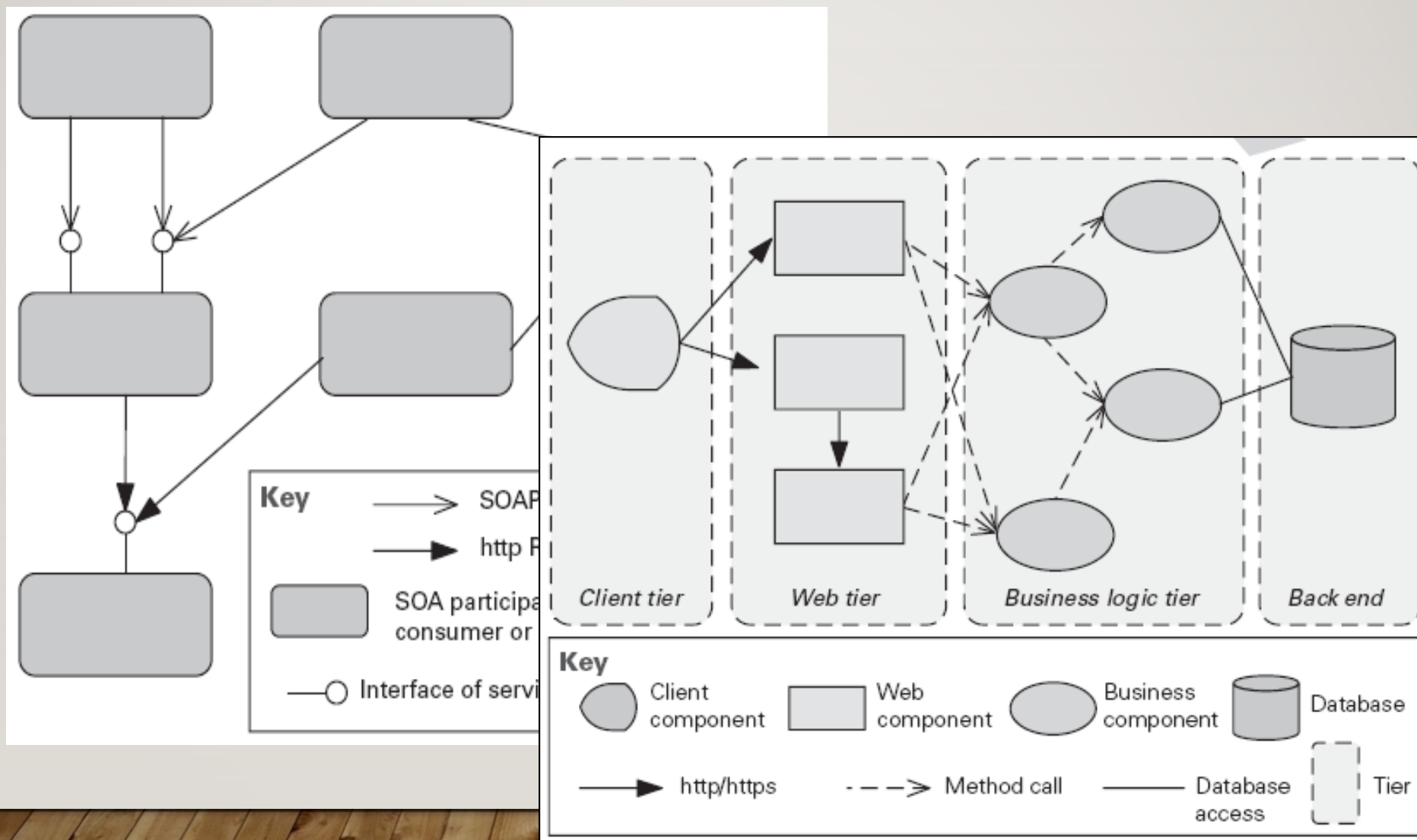
Element	Description
Thick client	Represents a MS.NET windows application developed using C# that connects to the server with the server-side components via web services (SOAP).
Portal web page	TOOD
SOAP + ATTA services (server)	This component represents the Apache Axis SOAP engine, which runs as a servlet. The Axis servlet takes http requests that point to a URL that is a web service http request. This request is converted to a SOAP request. In addition to the Axis servlet, there are two other servlets used by Axis (not depicted). Thus, an Axis server consists of three core web components: - AxisServlet – main servlet responsible for dispatching requests (org.apache.axis.transport.http.AxisServlet) - AdminServlet – serves the Administration console application (org.apache.axis.transport.http.AdminServlet) - SOAPMonitorService – monitor service (org.apache.axis.monitor.SOAPMonitorService)
Facade SLSB	A struts-like session bean that implements "business logic" functionality. Some facade session beans access the database directly (using DAO classes) and others use entity beans.
Product Files	Static content files stored in the file system. Include gpg files, images, audio & video files.

- 3. Variability Guide
- 4. Architecture Background
 - 4.1 Design Rationale
 - 4.2 Results of Analysis and Experiments
 - 4.3 Assumptions and Constraints
- 5. Related Views

MODALIDAD OVERLAY



MODALIDAD JERÁRQUICO



HAIKU

- Requiere que el lector entienda el contexto
- Y tener un **vocabulario compartido**

Project Lionheart is a publicly available web application that will help the Springfield Office of Management and Budget manage the city's requests for proposals (RFPs) and local businesses to find RFPs of interest.

Business Goals

- Reduce procurement costs by 30%
- Improve city engagement with local businesses
- Cut the time required to publish a new RFP in half

Top Quality Attributes

Security > Availability > Performance

Architecture Patterns Used

Service-Oriented Architecture (SOA), Layered web application, REST APIs for web services

Key Decisions and Rationale

- Node.js for web app—team has experience
- MySQL database—free, open source
- Apache Solr—free, open source
- SOA with REST—decouple components, team interested in experimenting with emerging tech trends
- Java for web services—open source, low-risk, great tool support

OTRO EJEMPLO DE HAIKU

Velocity Monitor

The purpose of the Velocity Monitor is to help Velocity administrators who are in charge of multiple collections distributed across multiple servers identify when a collection has failed on a specific server. A failure might occur for any number of reasons from an unexpected hardware failure to a defect in the Velocity configuration.

"What collection on what server failed?"

Technical Constraints

Velocity Monitor must run on Linux.

must work w/ Velocity 4.8.0.23

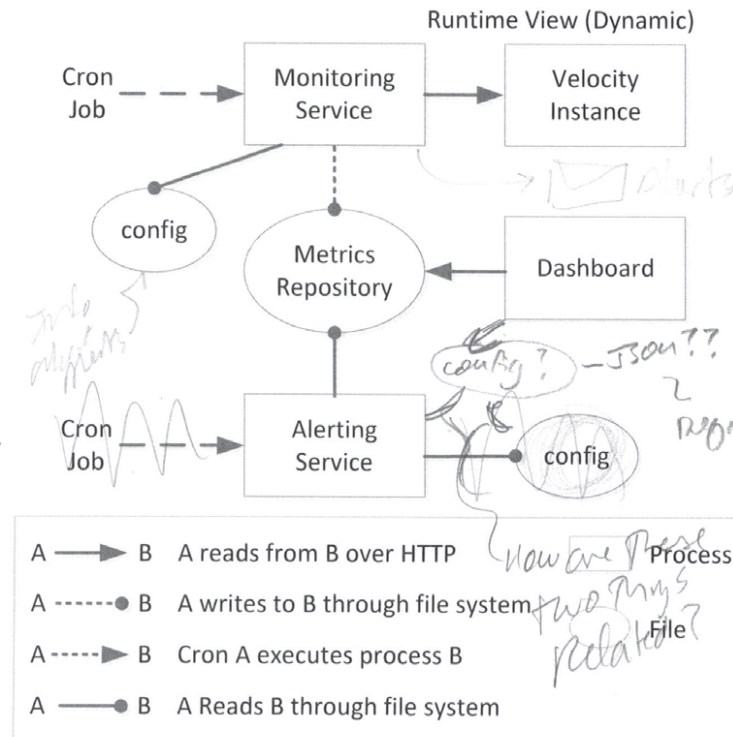
High-level Functional Requirements

As a KP IT professional I can monitor multiple Velocity instances for failure status with both a quick up/down status and details about the status.

As an IT professional I can receive an email alert when a Velocity instance fails. The email should include information related to the failure and a link to a dashboard for comprehensive, detailed information.

Design Decisions

Divided into small processes, each with specific responsibilities to build, maintain, and configure. Processes communicate with each other. Monitoring Service is the only "writer". A file is used to store data. It should be taken not to preclude a DB in the future. More data to be collected beyond what SNMP provides. Maintainability is generally

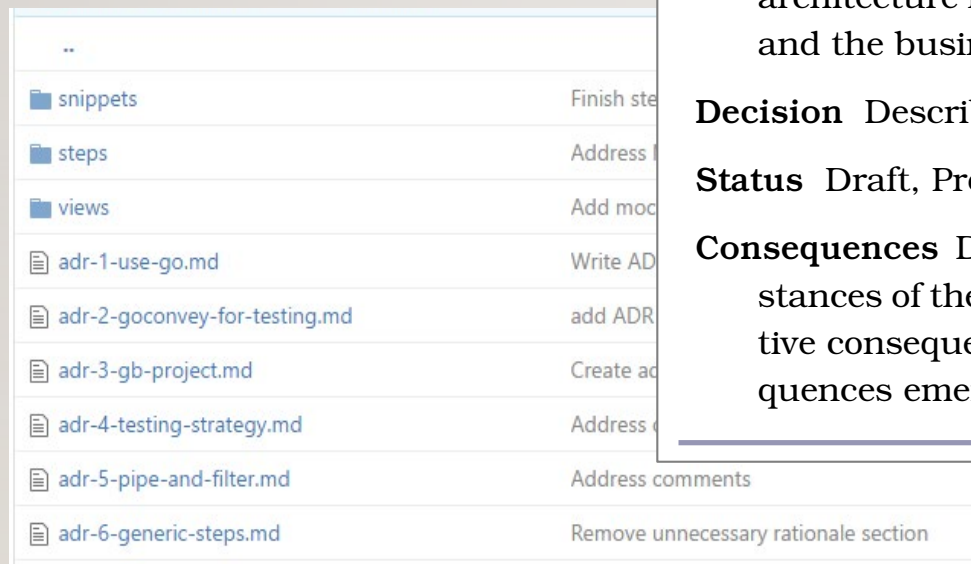


Responsibilities

- **Monitoring Service** collects data from Velocity using the REST Velocity API. Desired collections, servers, and deploy groups defined in config file.
- **Dashboard** is simple HTML/JavaScript application which displays data and metrics with context (red/green). Simple JS timer keeps this up to date.
- **Alerting Service** sends email to addresses defined in config file. Uses connection settings from config file.
- **Cron jobs** are set up to periodically run the monitoring and alerting services. Each can be configured separately to allow for more data to be collected, but not necessarily acted on via alerts.
- **Metrics Repository** is a simple file holding collected data over time. This might be many files or one. The exact format should be easily consumed by the Dashboard and Alerter, and easily writable by the Monitor.

ARCHITECTURE DECISION RECORDS

- Se persisten como parte del código fuente
- En formato Markdown



Title Include the ADR number and a descriptive title.

Context Explain the circumstances under which the decision was made. This should be a series of simple, factual statements. Describe relevant architecture influencers such as technology, skills, previous decisions, and the business or political climate among others.

Decision Describe the decision you made.

Status Draft, Proposed, Accepted, Superseded, or Deprecated

Consequences Describe how your decision will or has changed the circumstances of the system, stakeholders, and team. Both positive and negative consequences should be included. Update this section as consequences emerge and are understood.

EJEMPLOS DE ADRs

ADR 7: Public GitHub and Travis CI

Status: Proposed

We will use GitHub and Travis as our version control and continuous integration systems. All team collaboration will be conducted openly using GitHub systems.

ADR 1: Use the pipe-and-filter library

This ADR is closely related to [ADR 2](#).

In the past while writing agents, we decided it was a good idea to use the pipe and filter architectural pattern. See [this ADR](#) for context (if it's still around).

We also found that when the engineers who like pipes and filters took their eyes off of a project, changes would creep in that violated the pattern. Eventually this defeated the purpose of trying to structure the code in an architecturally evident way.

Joe has written a [pipe and filter library](#) that will soon be open sourced by the team. Using a library like this enforces the architectural style on the project: there are few ways to code around the framework short of ditching it altogether. This stops engineers from accidentally violating key assumptions on accident.

Decision

We will use the pipe-and-filter library to organize all application logic for the training agents.

Status

Accepted

Consequences

All training agents must fit within the context of pipe and filter. For the moment, we think that this is the case. There may be some pipelines that are very small, but will still work.

All filters must be completely decoupled from each other.

Concurrency can be controlled on a per-filter basis. This means we can write faster programs, but we also must be careful to write thread-safe filters if we want to run them concurrently.

Reacting to errors or expected failure modes will be much easier, and can be implemented with the message subscriber feature of the pipe and filter library.

to be released as open source. Travis CI is free and open source. Training practices are expected to assist in community familiar with GitHub workflow and tools.

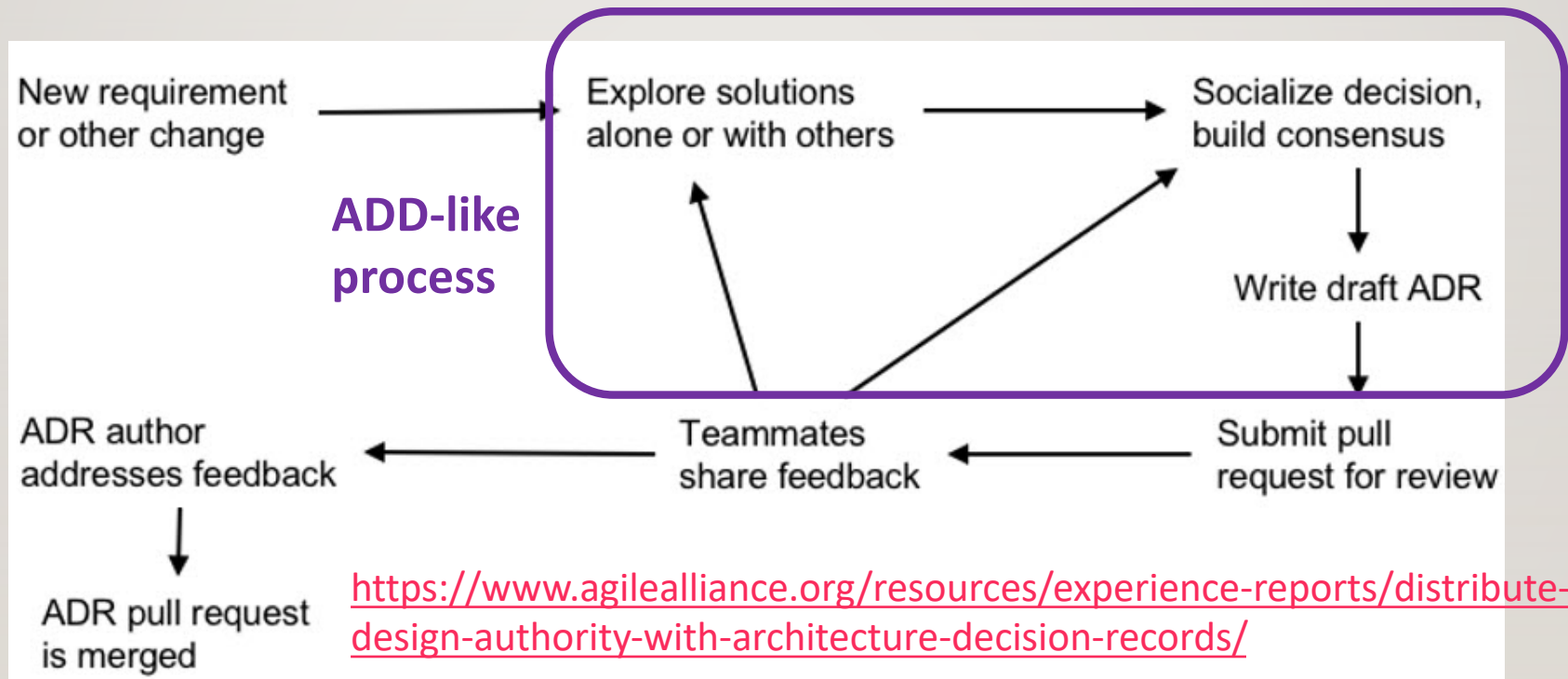
and edit code and documents (plain text).

officials is decreased as they become comfortable

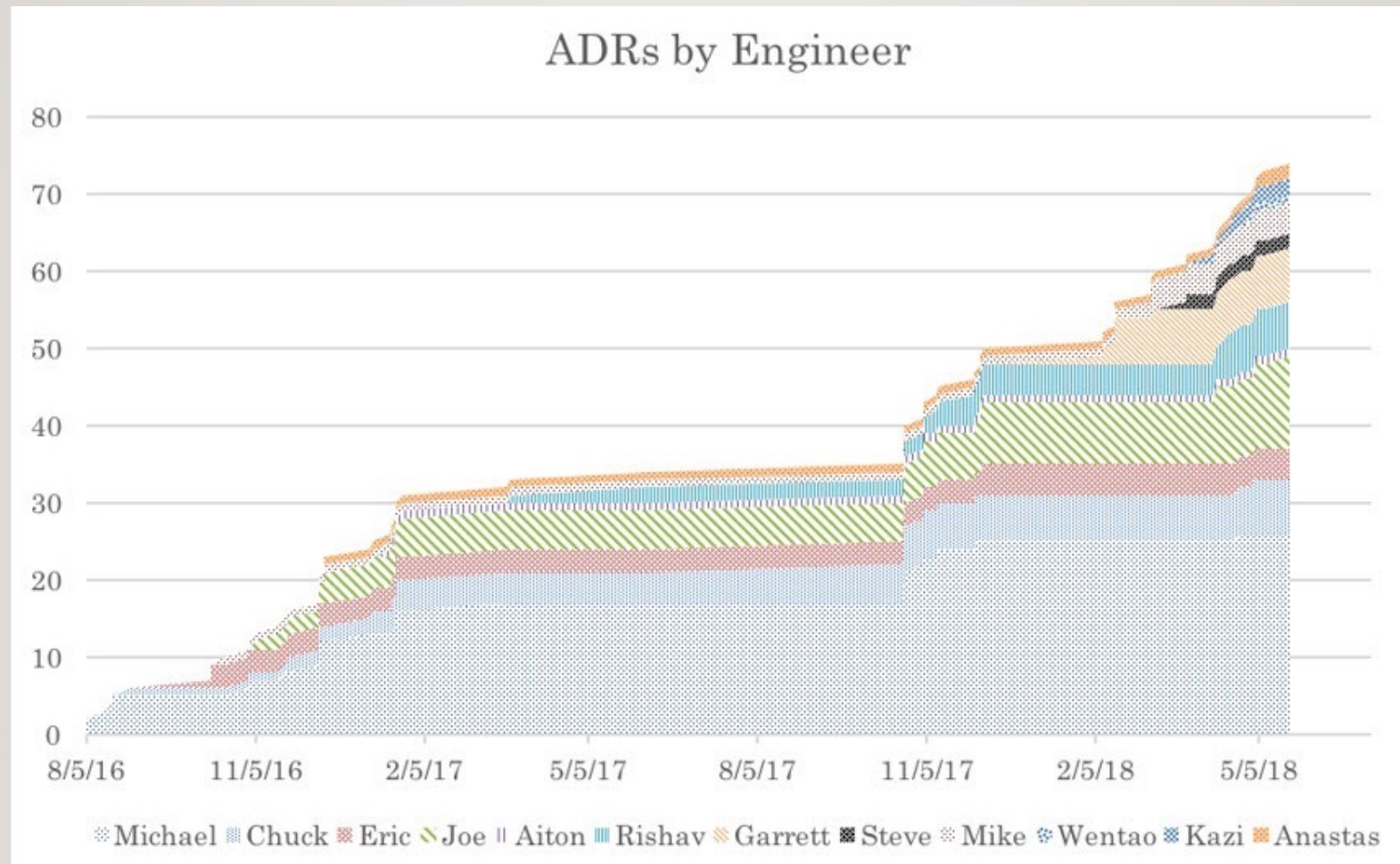
on are great, creating and storing diagrams is still

DECISIONES DESCENTRALIZADAS

- La descentralización no es “gratis”, requiere un “champion” y progresivamente que las capacidades arquitectónicas se propaguen a todo el equipo (por ej., desarrolladores)



EVOLUCIÓN DE DECISIONES



HLD – ENTRE HAIKU Y ADRs

- Puede utilizarse un template (por ej., 1-2 páginas) o algún otro mecanismo
 - Lo importante es capturar las decisiones y sus consecuencias
 - Suele acompañarse con alguna vista de arquitectura
 - También se capturan los puntos “abiertos/pendientes” de la sesión (issues)

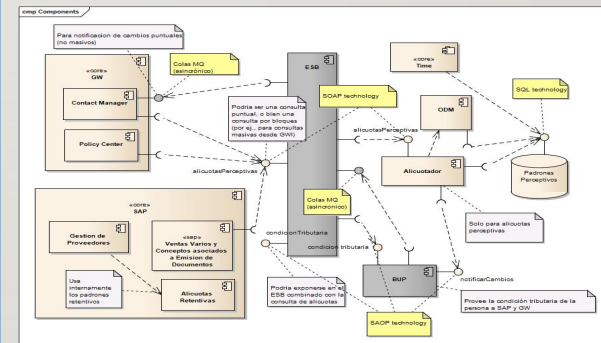
HLD Name

1. Description

- Domain & functional context
[Links to other related HLDs]

2. Solution Design

Design Decision	Rationale	Design Concerns
AD 1.1		
AD 1.2		
AD 1.3		



List of assumptions:

- assumption 1
- assumption 2

DISEÑOS DE ALTO NIVEL (HLD)

- Una **HLD** es una pieza de **conocimiento arquitectónico** que captura un **conjunto de decisiones de diseño** relacionadas para **una parte del sistema**, en un formato conciso
 - Describe uno o mas features de arquitectura, involucrando atributos de calidad y otros aspectos de diseño
 - No es diseño detallado
 - A menudo afecta varias historias de usuario
 - Debiera ser un documento “vivo”

HLD Template

1. Description
2. Solution Design
3. Quality-attribute analyses
4. Issues

PARTES DE UNA HLD (I)

HLD Name

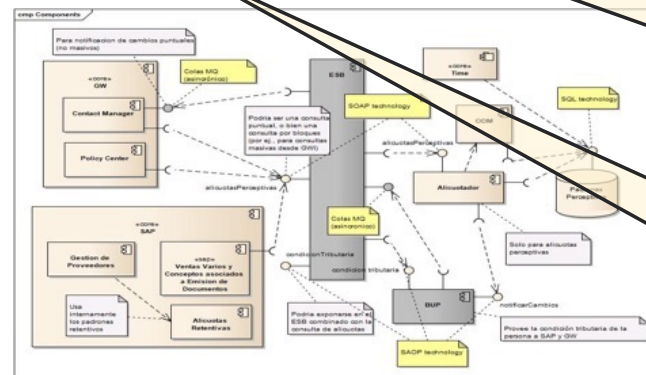
- Descripción
 - Principales requerimientos operacionales o contexto funcional
 - 3-4 párrafos
- Solución
 - Vista(s) arquitectónica(s)
 - Decisiones de diseño
 - Racional y otros aspectos
 - **Suposiciones**
 - 1-2 páginas

1. Description

- Domain & functional context
- [Links to other related HLDs]

2. Solution Design

Design Decision	Rationale	Design Concerns
AD 1.1		
AD 1.2		
AD 1.3		



List of assumptions:

- assumption 1
- assumption 2

AD: The ERP stores the information about payment methods, and it is the data master (for that information) with respect to other systems. The information is encrypted for security reasons

R: The ERP handles all the payments. Cohesion is improved. The ERP offers already a set of common services

AD: The ESB exposes a synchronous Web service to query for tax information about a person

PARTES DE UNA HLD (2)

- Si las suposiciones cambian, se debieran revisar las decisiones
- Atributos de calidad
 - Explicar cómo se ataca cada atributo, en base a evidencia
 - Ej. información de volumetría
- **Issues**
 - Aspectos no incluidos en la solución actual
 - Ej., falta de información de QAs
 - Otros aspectos
 - Ej., prueba de concepto

2. Solution Design

List of assumptions:

- *assumption 1*
- *assumption 2*

AA: The legacy core sends payment orders without encrypting customers' bank information, while the new core handles that information encrypted

3. Quality-attribute Analysis

Reliability

Performance

Interoperability

Security

...

AA: The internal token associated to a payment does not expire if the (same) credit card is renewed

4. Issues

- *issue 1*
- *issue 2*
- ...

I: Performance QoS of external provider is unknown

ALGUNAS LECCIONES APRENDIDAS SOBRE HLDs

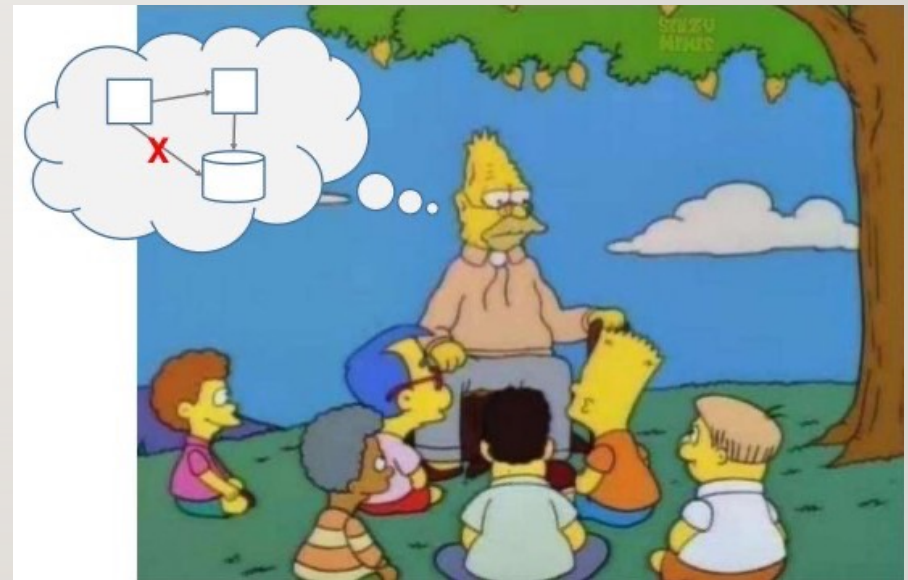
- Un buen mecanismo para capturar y comunicar la arquitectura a los stakeholders (no necesariamente desarrolladores)
- Aprovechamiento de los "issues" para gestionar deuda técnica
- Desafíos
 - Los consumidores de las HLDs las consideran "artefactos informativos" más que prescriptivos
 - Los desarrolladores o analistas pueden tomar decisiones adicionales (que incluyo contradigan una HLD) y no comunicar estas decisiones al equipo de arquitectos
 - Resistencia a incorporar versiones refinadas de una HLD, una vez que la implementación ya se encuentra avanzada
 - Elicitación de información de atributos de calidad
 - Combinación de varias HLDs "parciales" en un diseño "global"

RESUMEN

- La **captura** de las decisiones de arquitectura y de los issues correspondientes es clave
- Un desafío es la **gestión de la deuda técnica**
- ¿Cuáles de estas ideas/técnicas son factibles de implementar en el contexto de un proyecto?
 - Estrategia de dosificación



INTERCAMBIO



J. Andrés Díaz Pace

andres.diazpace@isistan.unicen.edu.ar